

CODE SPORT



Sandya Mannarswamy

In this month's column, we will discuss a set of interview questions.

We have been analysing many interview questions over the last couple of columns and we will continue to do so this month too. We will cover a couple of algorithmic questions in detail.

Let us look at a simple, yet well-known problem — the two-sum problem. Given an array A of N integers and an integer value K , check if there are two elements $A[i]$ and $A[j]$ such that $A[i] + A[j] = K$. Return 'True' if there are two such array elements. Else, return 'False'. While there are many variants to this problem, this is the basic one. As is our practice, we start by looking for the brute force solution. We can consider all possible pairs of (i, j) and check if any of them satisfies the above condition. This will lead to a $O(N^2)$ time complexity with a constant space complexity. Now, how can we improve this solution?

One way to rewrite the condition is in the following manner: If we are at array index 'j', is there a prior array element we have traversed, namely, 'a' such that $A[j] = K - a$? A simple linear search for checking this condition would result in $O(N)$ for each index 'j' leading to an overall time complexity of $O(N^2)$ again. So how do we avoid the linear search? The common pattern is to use some kind of hash map data structure, whereby we can check for this condition in $O(1)$ instead of $O(N)$. We do a linear scan over the array elements, where for each element $A[j]$, we first check if there exists a prior element $A[i]$ such that $A[j] = K - A[i]$. If it exists, we return 'True'. Else, we record the current element in the hash map. This results in a $O(N)$ time complexity with $O(N)$ additional space needed

for storing the hash map. Can we trade off the extra space complexity by sacrificing the $O(N)$ time complexity?

If we have a sorted array of integers, we can do a two-pointer approach to see if there exists a pair of elements that satisfy the condition. Sorting the array will require $O(N \log N)$. Then we maintain two pointers — the left pointing to index 0 of the sorted array and the right pointing to the last element of the sorted array. We check the sum of the elements pointed to by the left and right elements. If the sum is equal to K , we have found a valid pair of indices that satisfy the condition. If the sum is greater than K , we need to move down the right pointer to check if we can find a suitable pair. If the sum is less than K , we need to move up the left pointer. Here is the pseudo code for this approach:

```
bool twosum(int nums[], int N, int K)
{
    sort(nums.begin(), nums.end())
    int left = 0
    int right = N-1
    while (left < right)
    {
        int sum = nums[left] +
nums[right];
        if (sum == K)
            return true;
        else if (sum > K)
            right--;
        else
            left++;
    }
    Return false;
}
```

